

MATLAB to Python Deep Learning Pipeline for Signal & Image Processing

Project Overview

This project demonstrates an end-to-end engineering workflow that transitions from **classical signal and image processing techniques in MATLAB** to **Python-based numerical analysis and deep learning–driven super-resolution**.

The work emphasizes **quantitative evaluation, cross-platform consistency, and engineering reasoning**, rather than black-box model usage. The project was developed as a **technical portfolio sample** suitable for research, freelancing, and AI evaluation roles.

Key Objectives

- Apply classical image and signal processing methods using MATLAB
- Translate and validate equivalent workflows in Python
- Implement and evaluate a deep learning–based super-resolution model
- Compare numerical performance using standard quality metrics
- Demonstrate structured, reproducible, and explainable engineering practice

Skills & Technologies Demonstrated

Programming & Tools

- **MATLAB** (signal & image processing)
- **Python 3**
- **PyTorch**
- **NumPy, SciPy**
- **OpenCV**
- **scikit-image**
- **Matplotlib**

Technical Domains

- Signal processing (filtering, FFT concepts)
- Image preprocessing and enhancement
- Noise modeling and denoising
- Super-resolution using CNNs (SRCNN)
- Quantitative image quality evaluation
- Debugging numerical differences across platforms

Project Structure

project_root/

```

|
|
| — data/                # Input and processed image data
|   | — sample_gray.png    # Original grayscale image
|   | — HR/                # High-resolution images (ground truth)
|   |   | — LR/            # Low-resolution images (downsampled)
|   |
|   | — matlab_pipeline/    # MATLAB preprocessing & classical image processing
|   |   | — main_processing.m  # Full pipeline: noise addition, filtering, enhancement
|   |
|   | — results/           # MATLAB output images and metrics
|   |   | — histogram.png
|   |   | — noisy.png
|   |   | — filtered.png
|   |   | — enhanced.png
|   |   | — edges.png

```

- | └─ metrics_output.txt # MATLAB metrics: MSE, PSNR, SSIM
- |
- | └─ python_pipeline/ # Python translation of MATLAB pipeline
- | └─ prepare_data.py # Convert grayscale image to LR/HR, save in folders
- | └─ denoise.py # Bilateral / Gaussian filtering
- | └─ enhance.py # Contrast enhancement & edge detection
- | └─ evaluate.py # Metrics calculation: MSE, PSNR, SSIM
- | └─ results/ # Python output images and metrics
- | └─ metrics_output.txt # Python metrics comparison
- |
- | └─ deep_learning/ # Deep Learning super-resolution (SRCNN)
- | └─ srcnn.py # SRCNN model definition
- | └─ dataset.py # PyTorch dataset loader for HR/LR images
- | └─ train.py # Training script for SRCNN
- | └─ evaluate.py # Evaluation script (PSNR, SSIM, visualize results)
- | └─ results/ # Trained model & output images
- | └─ srcnn.pth # Trained SRCNN weights
- | └─ sr_output/ # Super-resolved images
- | └─ metrics_output.txt # Evaluation metrics
- |
- | └─ PDF_Report/ # Professional report for sharing with companies/clients
- └─ MATLAB_to_Python_DL_Pipeline.pdf

Workflow Description

1. MATLAB-Based Image Processing

- Image normalization and histogram analysis
- Noise injection and filtering (Gaussian, bilateral filtering)
- Edge detection and contrast enhancement
- Performance evaluation using:
 - Mean Squared Error (MSE)
 - Peak Signal-to-Noise Ratio (PSNR)
 - Structural Similarity Index (SSIM)

This phase establishes a **baseline understanding of image behavior**.

2. Python Reimplementation & Validation

- Replication of MATLAB processing using Python libraries
- Analysis of numerical differences between MATLAB and Python
- Parameter tuning to understand metric sensitivity
- Emphasis on **engineering validation**, not blind matching

This phase demonstrates **cross-platform reasoning and debugging**.

3. Deep Learning Super-Resolution (SRCNN)

- Generation of paired LR/HR datasets from a single grayscale image
- Implementation of SRCNN (Super-Resolution CNN) in PyTorch
- Training using pixel-wise MSE loss
- Evaluation using PSNR and SSIM
- Visual comparison of:
 - Low-resolution input

- Super-resolved output
- High-resolution reference

Evaluation Metrics

The following metrics are used consistently across MATLAB and Python implementations:

- **MSE (Mean Squared Error)**
- **PSNR (Peak Signal-to-Noise Ratio)**
- **SSIM (Structural Similarity Index)**

These metrics enable **objective performance comparison** between classical and deep learning approaches.

Key Engineering Insights

- Numerical results vary across platforms due to implementation differences
- Parameter sensitivity significantly affects image quality metrics
- Classical filtering can outperform naïve deep learning models in some cases
- Deep learning models require sufficient data and tuning to exceed classical baselines

This project emphasizes **understanding why results differ**, not just improving numbers.

Use Cases

- Technical portfolio for engineering or AI roles
- Freelance project sample (Upwork, Fiverr)
- Research or academic demonstration
- AI evaluation / tutoring roles requiring deep reasoning
- Foundation for advanced super-resolution or computer vision work

Possible Extensions

- Training with larger datasets
- Use of advanced SR models (EDSR, ESRGAN)
- Video super-resolution
- Conversion to Core ML for Apple devices
- Integration with scientific imaging pipelines

Author Note

This project was designed to highlight **engineering judgment, reproducibility, and analytical thinking**, rather than maximizing benchmark scores. It reflects real-world development conditions where understanding system behavior is as important as final performance.